

Project 03 예약 SaaS — Phase 1-2 학습정리

클린 아키텍처 골격 세우기: 프로젝트 4계층 분리 + 멀티프로젝트 Dockerfile

형식 변경: 이번 문서부터 "내가 한 것" 외에 "남들이 자주 겪는 문제 + 우리 케이스"를 함께 수록한다.

0. Phase 1-2는 무엇인가

Phase 1-1에서 DB(`reservationdb` + 전용 유저)를 준비했다. 1-2에서는 그 위에 올라갈 코드 구조를 클린 아키텍처 4계층으로 분리한다. 아직 기능(엔티티·인증)은 안 만든다. **방 구획부터 나누는 단계.**

1-2에서 한 일 요약 - `classlib` 프로젝트 3개 생성(Domain / Application / Infrastructure) + 기존 `Reservation_API` (Presentation) - 솔루션 등록 + **참조 방향 연결**(의존성이 안쪽을 향하게) - 빈 샘플 `Class1.cs` 정리, 빌드 검증 - **Dockerfile**을 **멀티프로젝트 대응**으로 재작성(루트로 이동) - `deploy.yml` 빌드 경로 수정 → 자동 배포로 검증

1. 왜 처음부터 4계층으로 나누나

단일 프로젝트로 인증·멀티테넌시까지 만든 뒤 나누면, 이미 얽힌 의존성을 풀어야 한다(집 다 짓고 벽 세우기). 처음에 칸을 나눠두면 "이 코드는 어느 칸?"을 매번 생각하게 되어 자연스럽게 깨끗해진다. 또 Dockerfile·배포 경로를 1-B에서 한 번 정리하면 이후 Phase에서 안 건드린다.

단, 실용적으로: 계층(프로젝트)은 4개로 나누되, 핵심 로직(멀티테넌시·낙관적 잠금)만 제대로 계층을 타고, 단순 CRUD는 가볍게 둔다.

2. 프로젝트 종류: classlib vs webapi

종류	명령	성격	우리 계층
클래스 라이브러리	<code>dotnet new classlib</code>	실행 안 됨. 다른 프로젝트가 참조하는 코드 묶음(도서관)	Domain / Application / Infrastructure
웹 API	<code>dotnet new webapi</code>	실행되는 앱(가게)	Reservation_API (Presentation)

안쪽 3개는 도서관(라이브러리), 바깥 1개만 가게(실행 앱). 이 구분이 클린 아키텍처의 물리적 표현.

3. 단계별 — CLI 방법 (실제 진행)

STEP 1. 프로젝트 4개 생성

```
dotnet new classlib -n Reservation.Domain
dotnet new classlib -n Reservation.Application
dotnet new classlib -n Reservation.Infrastructure
# Presentation은 기존 Reservation_API(webapi)를 사용 - 새로 만들지 않음
```

- `dotnet new classlib` : 클래스 라이브러리 생성.
- `-n` 이름 : 프로젝트 이름. **점(.) 구분 네이밍** (`Reservation.Domain`)은 .NET 관례 — 네임스페이스가 깔끔하고 같은 제품 계층으로 묶여 보인다.

STEP 2. 솔루션에 등록

```
dotnet sln add Reservation.Domain
dotnet sln add Reservation.Application
dotnet sln add Reservation.Infrastructure
```

- `dotnet sln add <프로젝트>`: 솔루션 바구니에 등록. 등록해야 VS에서 함께 보이고 한 번에 빌드된다.

STEP 3. 참조 연결 (★ 핵심)

```
# Application → Domain
dotnet add Reservation.Application reference Reservation.Domain
# Infrastructure → Application, Domain
dotnet add Reservation.Infrastructure reference Reservation.Application
dotnet add Reservation.Infrastructure reference Reservation.Domain
# API → Application, Infrastructure
dotnet add Reservation_API reference Reservation.Application
dotnet add Reservation_API reference Reservation.Infrastructure
```

- `dotnet add A reference B` = "A가 B를 참조한다(A는 B를 안다)".
- 의존성 방향:

```
Domain ← Application ← Infrastructure
          ↑
      Reservation_API ———
```

- ★ Domain은 아무것도 참조하지 않는다. `dotnet add Reservation.Domain reference ...` 가 하나도 없는 게 의도. → Domain에서 `using Microsoft.EntityFrameworkCore;` 를 쓰려 하면 **컴파일 에러**(참조가 없어 EF Core가 안 보임). "규칙을 어기고 싶어도 못 어기는" 구조.

STEP 4. 빈 샘플 정리 + 빌드

```
Remove-Item Reservation.Domain\Class1.cs
Remove-Item Reservation.Application\Class1.cs
Remove-Item Reservation.Infrastructure\Class1.cs
dotnet build
```

- `classlib` 생성 시 떨어오는 빈 `Class1.cs` 삭제. `dotnet build` 로 4계층이 올바르게 엮였는지 검증(`Build succeeded`).

4. 단계별 — GUI 방법 (Visual Studio 2026)

CLI와 1:1로 대응. 실무에선 돌을 섞어 쓴다.

작업	CLI	GUI (VS 2026)
프로젝트 생성	<code>dotnet new classlib -n 이름</code>	솔루션 우클릭 → 추가 → 새 프로젝트 → Class Library → 이름·위치(솔루션 폴더)· .NET 10
솔루션 등록	<code>dotnet sln add 프로젝트</code>	(새 프로젝트로 만들면 자동 등록) / 기존 것은 솔루션 우클릭 → 추가 → 기존 프로젝트
참조 연결	<code>dotnet add A reference B</code>	A 프로젝트의 종속성(Dependencies) 우클릭 → 프로젝트 참조 추가 → B 체크
빌드	<code>dotnet build</code>	Ctrl+Shift+B

GUI 참조 연결 직관: "A가 B를 참조" = **A의 종속성에서 B를 체크**. 화살표 시작점(A)에서 작업. - Application 종속성 → Domain 체크 - Infrastructure 종속성 → Application, Domain 체크 - Reservation_API 종속성 → Application, Infrastructure 체크 - **Domain → 아무것도 체크 안 함**

GUI 장점: 새 프로젝트를 솔루션 우클릭으로 만들면 등록이 자동. 참조도 체크박스라 한눈에 보이고, **순환 참조**를 시도하면 VS가 막아준다. **CLI 장점:** 한 번에 골격을 세우고 스크립트로 재현·문서화 가능.

5. 멀티프로젝트 Dockerfile (Phase 0 → 1-B 변화)

프로젝트가 1개 → 4개가 되면서, "API 하나만 빌드"하던 Dockerfile을 "솔루션 전체 빌드, 시작점은 API"로 바뀌어야 한다.

항목	Phase 0 (단일)	Phase 1-B (멀티)
Dockerfile 위치	Reservation_API/Dockerfile	솔루션 루트
빌드 컨텍스트	Reservation_API 폴더	솔루션 루트 전체
restore/publish	Reservation_API.csproj 하나	솔루션 restore + API 프로젝트 publish(참조까지 자동)
deploy.yml	-f Reservation_API/Dockerfile Reservation_API	-f Dockerfile .

```
# ==== build ====
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
# 솔루션 + 각 csproj만 먼저 복사 (레이어 캐싱)
COPY Reservation_Saas.slnx .
COPY Reservation.Domain/Reservation.Domain.csproj Reservation.Domain/
COPY Reservation.Application/Reservation.Application.csproj Reservation.Application/
COPY Reservation.Infrastructure/Reservation.Infrastructure.csproj Reservation.Infrastructure/
COPY Reservation_API/Reservation_API.csproj Reservation_API/
RUN dotnet restore
COPY . .
RUN dotnet publish Reservation_API/Reservation_API.csproj -c Release -o /app/publish

# ==== runtime ====
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish .
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "Reservation_API.dll"]
```

핵심 포인트 - csproj 4개를 각각 먼저 복사 → restore: 레이어 캐싱(소스보다 의존성 목록이 덜 바뀔)을 멀티프로젝트로 확장. 각 COPY A/B.csproj A/ 는 폴더 구조를 유지해야 프로젝트 참조가 풀린다. - dotnet publish 에 API 프로젝트를 명시: 실행 가능한 시작점은 API뿐. 참조하는 Application·Infrastructure·Domain은 자동으로 함께 빌드됨. - runtime 단계는 Phase 0과 동일: publish 결과물 안에 4개가 합쳐져 있으므로 ENTRYPOINT(Reservation_API.dll)도 그대로. - .dockerignore 도 루트로 이동, .git/ .github/ Doc/ 까지 제외(컨텍스트가 루트 전체라 불필요한 것이 늘어남).

6. ★ 흔한 함정 (남들이 자주 겪는 것 + 우리 케이스)

#	흔한 실수 (일반)	증상	우리 케이스 / 예방
1	순환 참조 (A→B, B→A)	빌드 실패 circular dependency	우리는 안 겪음. 의존성은 한 방향(안쪽)만. GUI는 체크박스를 막아줌
2	Domain에서 EF Core 등 참조	계층 오염(클린 아키텍처 깨짐)	Domain에 참조를 아예 안 걸어 컴파일 단계에서 차단
3	Dockerfile 경로/컨텍스트 불일치	로컬은 되는데 배포 빌드 실패	Dockerfile을 루트로 옮기고 deploy.yml도 -f Dockerfile . 로 동기화
4	FROM ... as (소문자)	FromAsCasing 경고	AS (대문자)로 통일. 빌드 로그가 줄 번호를 알려줌(line 18)
5	csproj 복사 누락	restore 단계에서 project not found	4개 csproj를 폴더 구조 유지하며 모두 복사
6	설정/시크릿 파일을 git에 커밋	저장소에 민감/불필요 파일 노출	reservation-fed-cred.json 이 올라감 → git rm --cached + .gitignore 추가로 제거
7	PowerShell `n 줄바꿈 오용	.gitignore 에 nreservation-... 처럼 글자 그대로 들어감	`n` 은 큰따옴표에서만 줄바꿈. 작은따옴표는 글자 그대로. 간단한 텍스트 편집은 에디터로 직접 하는 게 안전
8	PowerShell 명령 오타	Select_String (언더스코어) → 인식 불가	PowerShell 명령은 동사-명사 (하이픈): Select-String , Get-Content

케이스 6 보충 — git에 올라간 파일 지우기 `` powershell git rm --cached reservation-fed-cred.json # 추적에서만 제거, 로컬 파일은 유지(--cached)

.gitignore 에 파일명 한 줄 추가 (에디터로 직접)

git add .; git commit -m "..."; git push # 다음 push에 GitHub에서 사라짐 `` 우리 파일은 진짜 비밀값(비번/키)이 아니라 OIDC 설정이라, 히스토리 완전 삭제까지는 불필요. 비번/키였다면 히스토리 제거 (git filter-repo `) + 키 교체까지 필요.

7. IPlus(실무) vs 03 — 구조 비교

항목	IPlus (계층형)	03 (클린 아키텍처)
구조 단위	폴더로 구분 (한 프로젝트)	프로젝트로 분리 (4개)
계층	Controller / Service(IServices+Services) / Utils(DBManager)	Presentation / Application / Infrastructure / Domain
의존성 강제	폴더라 약속 수준 (컨트롤러가 DBManager 직접 호출 가능)	프로젝트 참조라 컴파일러가 강제
DB 결합	DBManager가 SqlConnection 직접 사용	Domain·Application은 DB를 모름

IPlus는 이미 인터페이스/구현 분리(IServices), DTO 분리(Request/Response), DB 접근 격리(DBManager)를 직관적으로 적용 중이었다. 03은 이를 **프로젝트 분리 + 의존성 역전으로 명시적·강제적으로 끌어올린 것**. 핵심 차이: "규칙을 사람이 지키느냐, 구조가 강제하느냐."

8. 면접에서 말할 수 있는 포인트

- 계층을 프로젝트로 물리 분리 — 폴더가 아닌 프로젝트 참조로 의존성 방향을 컴파일러가 강제. Domain은 EF Core를 참조조차 못 한다.
- 실용적 적용 — 골격은 4계층, 무게는 핵심에만. 단순 CRUD는 오버엔지니어링 회피. "어디까지 나눌지" 판단.
- 복잡도에 맞춘 배포 진화 — 단일 프로젝트 Dockerfile에서 멀티프로젝트(솔루션 restore + API publish + 레이어 캐싱)로 자연스럽게 확장.
- 레거시 → 모던 재설계 — 실무 계층형의 한계(약속 기반)를 이해하고 강제 기반 클린 아키텍처로 발전.
- 운영 위생 — 설정 파일을 저장소에서 분리(.gitignore), 빌드 경고까지 제거하는 습관.

9. 다음 — Phase 1-3 (엔티티)

골격이 섰으니, 이제 **Domain에 첫 엔티티**를 만든다. - **Tenant**, **User** (+ 이후 **Customer**, **Reservation**). - TenantId(멀티테넌시 기초), RowVersion/xmin(낙관적 잠금 기초) 자리 잡기. - 비즈니스 규칙을 엔티티에 캡슐화(예: 예약은 과거 시간 불가). - Domain부터(제일 안쪽, 의존성 없음) → 이후 1-D에서 EF Core로 Infrastructure에 연결.

Phase 1-2 학습정리 끝. 작성 기준: .NET 10 / Docker / VS 2026 / Azure Container Apps.